A Finding-Level Model for Internet Reachability under the FedRAMP VDR/VER Rules

Matthew Venne
Chief Technology Officer, stackArmor

July 2026

Abstract

Few evaluation steps carry more weight under the FedRAMP Vulnerability Detection and Response (VDR) and Vulnerability Evaluation and Reporting (VER) rules—launched on 2026-06-24 as part of the FedRAMP Consolidated Rules for 2026, and applicable to both 20x and Rev5 offerings—than deciding whether a vulnerability is internet-reachable. The Internet-Reachable Vulnerability (IRV) determination (`FRD-IRV`, `VER-EVA-EIR`) decides which findings land on the tightest `VDR-TFR-PVR` clocks and which are Not Internet-reachable Vulnerabilities (NIRV). Read maximally, FedRAMP's wording—any resource that ever processes internet-derived data hosts internet-reachable vulnerabilities—makes nearly every finding in a connected system IRV. A determination that answers "yes" for everything prioritizes nothing, and it buries the provider in per-finding documentation and remediation clocks the rules never intended. This paper proposes a **finding-level model** instead: reachability is computed, factor by factor, from an explicit equation—the delivery surface the internet can actually present to a component, intersected with the delivery surface the vulnerability actually requires—with every factor backed by evidence and defaulting to *reachable* whenever evidence is missing. Five evaluation gates compute the asset's exposure; transfer functions propagate the exposed surface through every routing, filtering, authentication, and sanitization hop; and the vulnerability's own trigger requirements finish the calculation. The model's practical center is sanitization: FedRAMP's own interception note lists *sanitize* among the preventions that remove IRV status, and most production applications already sanitize internet-sourced input. Verifying that sanitization is assessment work—the 3PAO collects the evidence through code review, static analysis, unit tests, and the web-application (DAST) scanning FedRAMP continuous monitoring already requires—and verified sanitization terminates reachability for the neutralized vulnerability classes downstream, concentrating scrutiny where it belongs: on the components that process data from unauthenticated users. Because that evidence stream is standing, the model converges in the steady state: for most reporting scenarios the internet-reachable set approximates the internet-accessible set. The convergence is conditional—providers must stand ready to re-prioritize and re-triage the moment threat intelligence surfaces a zero-day that defeats known sanitization techniques, WAF configurations, and other protections, which is the scenario FedRAMP's broad wording exists to govern. Where the evidence is not collected, the conservative default holds. Finally, we retain our Web Application Firewall (WAF) recommendation: FedRAMP permits treating perimeter-intercepted vulnerabilities as no longer internet-reachable, but the stricter posture is to keep them classified IRV and attest the mitigation via signed Vulnerability Exploitability eXchange (VEX)—noting the `VDR-TFR-KEV` carve-out that Known Exploited Vulnerabilities must still be remediated on CISA BOD 26-04 due dates even when fully mitigated.

1 Introduction

On 2026-06-24, FedRAMP launched the Consolidated Rules for 2026, including the Vulnerability Detection and Response (VDR) and Vulnerability Evaluation and Reporting (VER) rule sets. The rules apply to both FedRAMP 20x and Rev5 offerings, and they replace scanner-centric continuous monitoring with an evaluation-driven model. Providers MUST now evaluate every detected vulnerability three ways: is it an Internet-Reachable Vulnerability (IRV, `VER-EVA-EIR`)? Is it a Likely Exploitable Vulnerability (LEV, `VER-EVA-ELX`)? And what Potential Agency Impact N-rating (PAIN, `VER-EVA-EPA`) could exploitation carry? Together those three answers pick the row and column in the `VDR-TFR-PVR` timeframe matrix, and each must be reported per vulnerability under `VER-RPT-VDT`.

Getting reachability wrong is costly in both directions. Call too many internal flaws IRV, and the security team drowns in artificial emergencies on clocks the rules never meant to impose. Call a genuinely reachable flaw NIRV—FedRAMP's term for a Not Internet-reachable Vulnerability (`FRD-IRV`)—and an exploitable path stays open past its intended clock. `VER-EVA-ELX` adds a warning: evaluations made "recklessly or deliberately" wrong will damage a provider's FedRAMP Certification.

The hard part is that FedRAMP's definition, taken to the letter, does not discriminate. Section 2 argues that the maximalist reading—everything that ever touches internet-derived data is internet-reachable—is unworkable: it erases the prioritization the timeframe matrix exists to provide, and it sets documentation and remediation obligations no provider can carry. The determination has to be a *computation*, made per finding from evidence. Section 4 gives that computation as an explicit equation whose every factor is defined, evaluated by a named procedure, and defaulted to "reachable" when the evidence is missing. And one of those factors reflects something almost every production application already does: sanitize its internet-sourced input. FedRAMP's own rules name sanitization as a prevention that removes internet reachability—the model's job is to say exactly what evidence confirms it, and whose job it is to collect that evidence.

2 The Problem with "Everything Is Reachable"

FedRAMP defines an Internet-Reachable Vulnerability (`FRD-IRV`) as:

> "A vulnerability in a machine-based information resource that might be exploited or otherwise triggered by a payload originating from a source on the public internet."

The definition's notes push the scope wide: it explicitly "includes machine-based information resources that have no direct route to/from the internet but receive payloads or otherwise take action triggered by internet activity." The evaluation rule `VER-EVA-EIR` states the design intent directly: "FedRAMP focuses on internet-reachable (rather than internet-accessible) to ensure that any service that might receive a payload from the internet is prioritized if that service has a vulnerability that can be triggered by processing the data in the payload." Its notes supply two canonical examples—SQL injection reaching a private-network database through the application tier, and Log4Shell, "where exploitation was possible via vulnerable internet-reachable resources deep in the application stack that were often not internet-accessible themselves."

Read those words to the letter and follow the data. In a working cloud service, internet-derived data touches almost everything: the request flows to the application, the application writes to the database, the database replicates, events land on the message bus, workers consume the events, logs aggregate the lot. Under the maximal reading, every one of those components "might receive

a payload from the internet," so essentially every vulnerability anywhere in the connected system is IRV. The word *might* does the sweeping: it asks for possibility, not for a demonstrated path, and possibility is everywhere.

That reading is not practical, and the structure of FedRAMP's own rules says so. The `VDR-TFR-PVR` matrix (Table 1 shows Class C, likely the most common certification class) separates the IRV and NIRV columns precisely so that reachable findings move faster than unreachable ones—a 2× to 4× difference in the LEV rows. Declare everything IRV and that column collapses: the matrix degenerates to its leftmost clocks, and the determination FedRAMP asks providers to make per vulnerability (`VER-RPT-VDT`) becomes a rubber stamp that consumes evaluation and reporting effort while conveying nothing. Prioritization only exists if the classifier can say *no*.

PAIN Rating	**LEV + IRV**	**LEV + NIRV**	**NLEV**
PAIN-5	2 days	4 days	16 days
PAIN-4	4 days	8 days	64 days
PAIN-3	16 days	32 days	128 days
PAIN-2	48 days	128 days	192 days

Table 1: `VDR-TFR-PVR` mitigation/remediation timeframes for Class C certifications. Classes A and B share a more generous schedule on the same structure; Class D is tighter. The IRV/NIRV distinction is worth a factor of 2–4 on the clock—but only if some findings can actually earn NIRV.

The burden lands twice. Every IRV call must be evaluated, recorded, and reported per vulnerability under `VER-RPT-VDT`, so blanket reachability converts the reporting obligation into a treadmill covering the entire finding inventory. And every likely-exploitable finding in that inventory lands on the tightest clocks in its row—2 to 16 days at Class C—across the whole estate at once, a remediation posture no provider of realistic size can sustain. Nothing in the rules suggests they were designed to be unsatisfiable. `VER-EVA-ELX` says the quiet part aloud for the exploitability axis: "the simple reality is that most traditional vulnerabilities discovered by scanners or during assessment are not likely to be exploitable." The rules are built for discrimination—for sorting findings into different treatments—and a reachability determination that cannot discriminate defeats their design.

The way out is also in FedRAMP's own text. The `VER-EVA-EIR` notes continue: "the simplest way to prevent exploitation of internet-reachable vulnerabilities is to intercept, inspect, filter, *sanitize*, reject, or otherwise deflect triggering payloads before they are processed by the vulnerable resource; once this prevention is in place the vulnerability should no longer be considered an internet-reachable vulnerability." *Sanitize* is FedRAMP's word, on FedRAMP's list of preventions that remove reachability. And sanitization is not exotic: parameterized queries, schema validation, type and bounds checking, output encoding—this is what production applications already do to internet-sourced input before handing it downstream. The honest question for an assessment is therefore not "does internet data ever flow toward this component?"—in a connected system it nearly always does—but "is the sanitization the application already performs *verified*?" That verification is the auditor's job, and the evidence stream already exists: FedRAMP continuous monitoring requires regular web-application (DAST) scanning, and the CSP's development lifecycle produces code reviews, static-analysis results, and test suites as a matter of course. The 3PAO's task is to collect and weigh that evidence, not to invent a new documentation regime—and the scrutiny it buys should land where the risk concentrates: on the components that process data from *unauthenticated* users, the entry points and the pre-authentication surfaces they feed, rather

than uniformly across the estate. This paper's model (Section 4) makes reachability a per-finding computation in which verified sanitization is a first-class factor, every factor is evidence-backed, and every evidence gap defaults the finding to reachable. Run to its conclusion (Section 9), the model restores proportionality: with the standing evidence in place, the reachable set converges on the accessible set for everyday reporting—and expands back toward FedRAMP's full breadth precisely when threat intelligence says the known protections no longer hold.

One distinction stays load-bearing throughout, so we fix the vocabulary now. *Internet-accessible* describes an entry point: an asset that accepts traffic arriving directly from the public internet. *Internet-reachable* is FedRAMP's broader status: it applies to any vulnerability that internet-originated data can reach and trigger, whether its host is an entry point or sits downstream of one. Every internet-accessible asset hosts candidates for reachability; the reverse is nowhere close to true.

IN PLAIN TERMS

FedRAMP's definition, taken to the letter, would label nearly every flaw in a connected system "internet-reachable" — because in a real system, data that started on the internet eventually touches almost everything. A label that everything carries means nothing: it can't tell the security team what to fix first, and it puts the whole estate on the fastest deadlines at once. But FedRAMP's own rules also say that stopping the dangerous input before it reaches the flaw — including *sanitizing* it — makes the flaw not internet-reachable. Most applications already clean the input they pass along, and checking that is what audits are for: the web-application scans FedRAMP already requires in continuous monitoring, plus the code reviews and tests the development team already runs, are the proof. So: compute reachability per finding, let the auditor collect the sanitization evidence, put the extra scrutiny on whatever handles input from strangers who never logged in — and when the evidence isn't there, keep the cautious default.

3 Scope Realism: What the IRV Determination Actually Controls

Before presenting the model, it helps to be clear about the stakes, because the IRV determination alone does not set the tight clock. The `VDR-TFR-PVR` matrix is indexed by *three* evaluations at once: the PAIN rating picks the row, and the combination of likely exploitability and internet reachability picks the column (LEV + IRV, LEV + NIRV, or NLEV), as Table 1 showed. Three observations keep the reachability problem in proportion:

- **LEV gates the tight columns.** The IRV and NIRV columns only differ for vulnerabilities that are also likely exploitable, and `VER-EVA-ELX` is blunt that those are a minority: "the simple reality is that most traditional vulnerabilities discovered by scanners or during assessment are not likely to be exploitable." So reachability matters most for the LEV subset. Note, though, that reachability feeds the LEV call itself: `FRD-LEV` requires the vulnerability to be "reachable by a likely threat actor," and its floor note makes any vulnerability "that an automated unauthenticated system can exploit over the internet" automatically LEV.

- **The timeframes are should-level and are satisfied by mitigation.** `VDR-TFR-PVR` asks providers to "partially mitigate, fully mitigate, or remediate" *to a lower Potential Agency Impact N-rating* within the timeframe. A partial mitigation that provably lowers the PAIN rating satisfies the clock. (FedRAMP's definition of mitigation is broader than the rule's wording suggests: `FRD-PMV` covers reductions in likelihood *or* PAIN—Section 13 returns to this.) That does not end the process—the finding continues on the new, longer clock, and each reduction is reported under `VER-RPT-VDT`—but the descent is bounded. It ends in remediation, full mitigation, PAIN-1 (which carries no timeframe), or formal acceptance: `VER-TFR-MAV` requires any

vulnerability that will not be fully mitigated or remediated within 192 days of evaluation to be categorized as an accepted vulnerability. No single timeframe demands full remediation (with the KEV exception discussed in Section 13).

- **The goal is determinism, not deadline-shrinking.** This model does not exist to shrink the IRV list through aggressive interpretation. It exists to make each call deterministic, evidence-backed, auditable, *and proportionate*, so the LEV + IRV set a provider reports under `VER-RPT-VDT` can be defended line by line—to a 3PAO, an agency, or FedRAMP itself.

IN PLAIN TERMS

Reachability is one of three dials, and rarely decisive alone: the tightest clocks bind only flaws that are *also* likely exploitable — a minority — and each clock is satisfied by demonstrably knocking the risk down a rung, not only by a full fix. So the goal here is not to argue findings off the urgent list; it is to make every reachability call one you can prove to an auditor.

4 The Finding-Level Model

We tag each *asset* with its internet-reachability $R(a)$ as a first-class, reusable property—useful on its own for inventory and attack-surface work. But the remediation timeline is decided per *finding*: a pair (v, a), vulnerability v on component a, combining that asset tag with facts about the vulnerability itself.

Factored form (operational).

$$\begin{aligned}\mathrm{IRV}(v,a) \;&= R(a) \;\cdot\; \mathbb{1}[\mathrm{AV}(v) = N] \\ &\quad\cdot\; \mathbb{1}[\text{protocol/version of } v \text{ on the exposed surface}]\end{aligned}$$

IN PLAIN TERMS

A vulnerability gets the tight "internet-facing" clock only if *all three* are true: the machine is reachable from the internet, *and* the flaw can be attacked over the network at all, *and* the exact service/protocol the flaw lives in is actually one the internet can reach. Miss any one and it gets the slower "internal" clock. (The $\mathbb{1}[\cdots]$ is just "1 if true, 0 if false"; multiply them, and one zero makes the whole thing zero.)

- $R(a)$ — the **asset tag**: is the resource internet-reachable at all (computed by the gates of Section 5). If $R(a) = 0$, every finding on the asset is NIRV for the direct channel—the cheap short-circuit; you never open the CVE.
- If $R(a) = 1$, evaluate the internet-exposed remediation column *only* when the vuln is `AV:N` **and** its protocol/version actually rides the exposed surface. Otherwise that finding drops to the NIRV column even though the asset is tagged reachable.

Set form (why the two vuln factors are one test). Let D be a space of **delivery descriptors** — the "shapes" of traffic:

- $X(v) \subseteq D$ — the **required surface**: the delivery shapes under which v can actually be triggered (from the CVE).
- $E(a) \subseteq D$ — the **exposed surface**: the delivery shapes an unauthenticated internet actor can actually get *to component* a (from infra).

$$\mathrm{IRV}(v,a) \;=\; \mathbb{1}[\,E(a) \cap X(v) \neq \varnothing\,] \tag{1}$$

IN PLAIN TERMS

Picture two lists. **List** E is every door the internet can actually get to on this component — for example, "the web door is open to everyone." **List** X is every door this particular attack could use — for example, "this exploit only works through the SSH door." Now just lay the two lists side by side. If the *same door* shows up on both, the attacker has a way in — internet-reachable. If nothing matches — the internet can reach the web door, but the attack needs the SSH door, and that one isn't open — there's simply no way in, so it's not reachable. (That "nothing matches" is what $\cap = \varnothing$ means: no shared door.)

This is the same statement: the asset tag is exactly $R(a) = \mathbb{1}[E(a) \neq \varnothing]$ (the asset has *some* internet-deliverable surface), and the `AV:N` and protocol/version factors are "$X(v)$ meets that surface." A non-empty intersection already implies $R(a) = 1$, so the asset tag is the necessary first factor—not a discarded one. The set form is also what carries the transitive cases FedRAMP cares about: for a downstream component, $E(a)$ is not what the internet can *connect* to but what internet-originated *content* can still be delivered to it through the tiers in between—the subject of Sections 7 and 8. Every factor in either form defaults to *reachable* unless evidence says otherwise; that fail-safe is developed in Section 12.

5 Computing the Asset Tag $R(a)$: The Five Gates

The asset tag is not a single chain of ANDs. There are *several independent ways* to be exposed—directly on a public IP, or behind a public load balancer, or via a container ingress—and the asset is reachable if **any one** path is open. So $R(a)$ is an **OR across paths**, and each path is its own **AND** of conditions, checked *at that path's own enforcement point*:

$$R(a) \;=\; \mathbb{1}\Big[\; \exists\, P \in \text{paths}(a) :\; \text{reachable}(P) \;\Big]$$
$$\text{reachable}(P) \;=\; \text{route}(P) \;\wedge\; \text{open}(P) \;\wedge\; \neg\,\text{auth}(P) \;\wedge\; \text{live}(P)$$

A path P is *reachable* when all four conditions hold:

- route(P) — a network route physically exists to the asset (a public IP plus a gateway route, or a public load balancer that forwards to it); a wide-open firewall on a private, unrouted box is still unreachable;
- open(P) — its public entry point accepts traffic from sources beyond an enumerable, attributed set (Gate 2 gives the decision rule; breadth beyond a `/20` is the red flag that triggers review);
- $\neg$ auth(P) — nothing along the path forces the visitor to log in before the packet reaches the vulnerable component;
- live(P) — it ends at a port something is actually listening on.

Only the **public entry point** counts toward "open." A backend security group that merely lets the load balancer through is what makes the path *connect* (route/live)—it does not make the path narrow. The two common archetypes make the OR concrete:

$$R(a) \;=\; \underbrace{\Big[\; \text{pubIP}(a) \;\wedge\; \text{route}_{\text{IGW}}(a) \;\wedge\; \text{fwOpen}(a) \;\wedge\; \neg\,\text{edgeAuth}(a) \;\Big]}_{\text{direct: public IP on the host itself}}$$
$$\vee \quad \underbrace{\bigvee_{L \in \text{pubLB}(a)} \Big[\; \text{forwards}(L,a) \;\wedge\; \text{lbOpen}(L) \;\wedge\; \neg\,\text{edgeAuth}(L) \;\Big]}_{\text{via a public load balancer / ingress (no public IP or IGW route needed)}}$$

What the symbols mean:

- paths(a) — the set of internet-ingress paths that terminate at a (direct, one per fronting load balancer / ingress, etc.).
- pubIP(a) — the host itself has a routable **public IP**; and $\text{route}_{\text{IGW}}(a)$ — its subnet has an active default route to an **internet gateway**. *Both are required only for the direct path*—an LB-fronted backend typically has neither. Gate 1 evaluates them.
- $\text{fwOpen}(a) = \exists\, q :\ \text{listen}(a, q) \wedge \exists\, r \in \text{allow}(a, q) : \text{prefix}(r) < P$ — on the **host's own** firewall, some *listening* port is opened to a source wider than the threshold. (Size is the screening form; Gate 2 refines the decision to source *attribution*.)
- pubLB(a) — the public load balancers / ingresses that forward to a; forwards(L, a) — L is public-facing and routes traffic to a *live listening* port on a (for containers this is Gate 4: target of an `HTTPRoute`/`TCPRoute` on a public gateway, an `Ingress` on a public LB, or a public `NodePort`).
- lbOpen(L) — the **load balancer's own** source restriction is wider than the threshold (it is *not* IP-locked to an attributed set; same Gate 2 rule, evaluated at the LB).
- $P = 20$ — the **breadth threshold**. "Broader than a `/20`" means a bigger address block, i.e. a *smaller* prefix number, $\text{prefix}(r) < 20$.
- edgeAuth($\cdot$) — a bypass-free identity-aware proxy (Google IAP, AWS Verified Access, ALB OIDC/Cognito) that authenticates every request. Evaluated on the **host** for the direct path, and on the **load balancer** for the LB path. Gate 3 gives the qualifying conditions.

IN PLAIN TERMS

There is more than one way to be exposed, so this is an "OR." A box counts as internet-facing if *either*:
(A) it's directly on the internet — it has its own public IP, a route out, its own firewall opens a live port to a wide slice of the internet (wider than ∼4,000 addresses), and there's no login proxy on the host; *or*
(B) a public load balancer exposes it — some public LB forwards to a port the box is actually listening on, that LB isn't locked down to a small IP range, and that LB doesn't force a login first.
Note case (B) needs *no* public IP and *no* internet-gateway route on the box itself — which is why any single formula written only for case (A) misclassifies LB-fronted backends. And each "open?" and "login wall?" check is done where that path actually enforces it: on the host for (A), on the load balancer for (B).

The whole thing. The finding-level rule is the asset OR (some open path exists) ANDed with the vulnerability factors:

$$\text{IRV}(v, a) = \underbrace{\mathbb{1}\big[\exists\, P \in \text{paths}(a) : \text{reachable}(P)\big]}_{\text{asset reachable: } R(a) \text{ (OR across paths)}} \cdot \underbrace{\mathbb{1}[\text{AV}(v){=}N] \cdot \mathbb{1}[\pi(v) \text{ exposed}] \cdot \mathbb{1}[\nu(v) \text{ exposed}]}_{\text{the flaw rides that surface}}$$

IN PLAIN TERMS

The full checklist: *at least one* internet path into the box is open (OR across paths), *and* the flaw is network-attackable on a protocol/version that path actually delivers. If no path is open the first factor is zero; if the flaw isn't network-reachable the later factors are zero — either way the finding gets the slower internal-remediation clock. (Because a protocol can be delivered on one path but not another — e.g. HTTP/2 direct but downgraded through the LB — "π/ν exposed" means "on at least one *reachable*

path," which the surface union $E(a)$ handles automatically.)

The five gates that follow are the evaluation procedures for these predicates. Each gate is deterministic, names its evidence, and—like every factor in the model—defaults to *exposed* when the evidence is incomplete.

5.1 Gate 1: IP Address and Routing — Evaluating route(P)

Gate 1 evaluates pubIP(a) and $\text{route}_{\text{IGW}}(a)$, the two preconditions of the direct path. An asset is directly exposed at the network layer only if both hold:

1. The asset holds a routable public IP address, or belongs to a NAT or load-balancer pool that maps public traffic to it.
2. The subnet's routing table has an active route to an internet gateway (`0.0.0.0/0` via an IGW or equivalent).

Assets failing either condition—private and isolated subnets, egress-only NAT configurations—have $R(a) = 0$ on the direct path, and if no load-balancer path exists either, $R(a) = 0$ outright. That closes the *connection* channel only. Internet-originated *content* can still be relayed to such an asset by the tiers in front of it—the private-subnet database in FedRAMP's SQL-injection example fails this gate and can still host reachable injection flaws—which is exactly what the surface-propagation and sanitization analysis of Sections 7 and 8 decides.

5.2 Gate 2: Firewall Source Population — Evaluating open(P)

In the equations, open(P) appears as a breadth test—wider than a `/20`. Operationally, breadth is only the red flag; *attribution* makes the call. Even with a public IP and an internet route, an asset whose inbound rules restrict sources to specific known networks is not exposed to the general internet. The tempting way to formalize that is a size rule—exclude whenever every permitted source prefix is `/20` or smaller—but any size rule fails in both directions. It can be gamed: one hundred separate `/24` rules each pass the per-rule test while together covering more address space than a `/16`. And it flags safe configurations: a single allowlisted partner `/16` fails the test even though it represents exactly one known organization.

The gate therefore looks at each listening port, takes the *union* of every source range permitted by any firewall rule, security group, or WAF-layer IP restriction that governs it, and asks who owns that space:

> **Exclusion criterion.** The asset/port is excluded from direct exposure if and only if the union of permitted sources is enumerable and every constituent range is attributable to an identified organization under an agreement with the provider—corporate networks, named partners, or specific customer egress ranges on record.

Size survives only as a red flag. A union larger than a `/20`-equivalent (4,096 addresses) SHOULD trigger a review of the attribution records, because allowlists that big tend to accumulate ranges nobody can explain. But size only decides when to look harder, never the outcome: a fully attributed partner `/16` passes; a hundred anonymous `/24`s do not.

IN PLAIN TERMS

An allowlist earns an exclusion by being *accounted for*, not by being small. If every permitted address range can be traced to a named organization you have a relationship with, the door is closed to the general internet; if parts of the allowlist are anonymous, it is not — no matter how narrow the individual ranges look.

5.3 Gate 3: Remote-Access Boundaries — Evaluating $\neg\,\mathrm{auth}(P)$

This gate is the edgeAuth(·) predicate of the symbol legend, evaluated on the host for the direct path and on the load balancer for the LB path. Many architectures put backends behind a boundary that checks identity at the edge: an identity-aware proxy (Google Identity-Aware Proxy, AWS Verified Access, ALB OIDC/Cognito authentication) or a client VPN. Both reject unauthenticated traffic before the backend ever processes it. The boundary must be exactly that—a component *separate from the application*. An application's own login page never qualifies: the login form, credential parsing, password reset, and session issuance are unauthenticated internet input processed by the application itself, so the pre-authentication attack surface *is* the application. Yet for true edge components, convention treats the two kinds very differently: backends behind a VPN are habitually called "internal," while backends behind an IAP are often scored as exposed. This gate applies one functional test to both:

> *Edge authentication qualifies for exclusion from direct exposure when it functions as* ***remote-access control*** *for the organization's own people and tooling—the clientless-VPN role, fronting admin consoles, operations tooling, and internal dashboards. It does not qualify when it functions as* ***customer authentication*** *for the service itself. The difference is the boundary's role, not the size of the crowd behind it.*

Rejecting unauthenticated traffic before the vulnerable resource sees it satisfies the interception principle in `VER-EVA-EIR`: "the simplest way to prevent exploitation of internet-reachable vulnerabilities is to intercept, inspect, filter, sanitize, reject, or otherwise deflect triggering payloads before they are processed by the vulnerable resource; once this prevention is in place the vulnerability should no longer be considered an internet-reachable vulnerability." For *unauthenticated* traffic, an enforcing IAP or VPN is exactly that prevention. The question that remains is simple: *who can still deliver data after logging in?* For a remote-access boundary, the answer is the organization's own people. For customer authentication, the answer is the service's internet user base. In verifiable form:

> **Exclusion criterion.** A vulnerability v on backend asset a may be excluded from direct exposure via edge authentication if and only if all of the following hold:
>
> (a) the boundary rejects unauthenticated traffic before v's vulnerable code processes any attacker-controlled data;
>
> (b) the boundary's role is remote access—it fronts the organization's own personnel and tooling, not the customer-facing service. The role is a recorded assertion: the operator declares it and the assessor validates it, the same way an assessor confirms what any other control is actually doing. It is not a head-count. A change-management system behind an IAP at a large provider may have a hundred thousand authorized users and is still remote access; a customer portal with three hundred accounts is still the service's front door; and
>
> (c) the backend verifiably validates the boundary's cryptographic identity assertion (e.g., validating the IAP-signed JWT on every request), so that header spoofing or misrouting around the boundary does not silently void the exclusion; and
>
> (d) the backend authenticates its users *independently* of the boundary—a second login flow, the way a VPN user still signs into the console behind the tunnel. The boundary's signed assertion in (c) is bypass protection, not authentication. A backend that consumes the boundary's signed headers as its entire authentication

story is one hijacked edge session or one compromised proxy away from granting authenticated access—a compromised IAP will cheerfully mint valid signed headers to the backend—so such a backend is treated as directly exposed.

Condition (b) is where the usual intuitions succeed and fail. "Behind the VPN, therefore internal" quietly relies on it—and it is usually right, because VPNs and identity-aware proxies overwhelmingly front an organization's own tooling. But when (a) holds and (b) fails—the login wall is the front door of the customer application—internet data still flows through authentication and into the backend. SQL injection behind the customer login is still an IRV, whatever brand of proxy enforces that login. In that setup, edge authentication is an *exploitability* input, not a reachability exclusion: it defeats the `FRD-LEV` floor ("any vulnerability that an automated unauthenticated system can exploit over the internet is a likely exploitable vulnerability"), which may support an NLEV call under `VER-EVA-ELX`, but it does not make the data path disappear.

IN PLAIN TERMS

An IAP or VPN in front of your *staff* tools is a real boundary: it exists to let your own people in, so what sits behind it is not internet-reachable — no matter how many staff you have. The same kind of login as the front door of your *customer* product excludes nothing — anyone on the internet can hold an account and send hostile input straight through it. What matters is what the door is *for*, not the door itself or the number of keys. Two more rules keep it honest: the door must be a separate component, not the app's own login page — a login page is the app parsing stranger input — and the app behind the door still needs its own login, just as tools behind a VPN do. One door is never enough: if the app trusts the proxy's stamp as the whole login, whoever beats the proxy owns the app. (Either way, the perimeter itself — the proxy or VPN concentrator — is internet-accessible.)

In practice, the failing case is rare. Almost every identity-aware proxy in a FedRAMP environment sits in front of staff tools—the qualifying role. Putting an edge login in front of the customer application itself is uncommon. So condition (b) rarely costs a real deployment anything. Its job is to close a loophole: without it, any login page would make everything behind it "not internet-reachable" on paper, including a SQL injection that fires after login. No assessor would accept that, and it is the first thing one would test. That is also why the role call belongs to people, not to automation. A dedicated boundary component that provably rejects unauthenticated traffic is treated as remote access by default; the operator declares any customer-facing exception in governed metadata, and the assessor confirms during assessment that the declarations match what the service actually does—determining whether an auth proxy is performing customer-facing authentication is the assessor's job, as is confirming that the backend's own authentication under condition (d) is a real second flow and not a pass-through of the boundary's headers. Cloud signals can flag a declaration for review (a GCP IAP policy granting `allUsers`, for example, is worth a question), but they refine the audit; they do not make the determination. The companion tooling specification covers the details.

No matter how the boundary is configured, the perimeter itself is internet-accessible. The VPN server, the load balancer, the proxy—anything that touches traffic before the login check—is exposed by definition. A login wall cannot protect the thing that *is* the login wall. The two technologies differ only in how much sits in front of that check. A VPN turns strangers away before a connection ever reaches anything behind it, so the VPN server is the only exposed piece. An identity-aware proxy decides later: the cloud load balancer has already accepted the connection and parsed the request before the login check runs, so the load balancer and its parsing stack are exposed too. Behind the boundary, the exclusion works the same either way; what differs is which front-line components are internet-accessible.

5.4 Gate 4: Kubernetes and Container Routing — Enumerating pubLB(a)

Containers need their own check, because a pod's exposure is decoupled from its node's. For container estates this gate enumerates the load-balancer path family—the pubLB(a) and forwards(L, a) terms. A pod is a direct-exposure entry point if and only if it is the active target of at least one of:

(a) a Gateway API route (`HTTPRoute`, `TCPRoute`, etc.) bound to a public-facing Gateway;
(b) an `Ingress` bound to a public load balancer;
(c) a `Service` of type `LoadBalancer` with a public address;
(d) a `NodePort` service on nodes reachable from the internet; or
(e) `hostNetwork: true` or a `hostPort` mapping on a node with public exposure—this includes DaemonSets, which frequently run with host networking on every node and are easy to overlook in route-centric inventories.

Pods exposed only through internal load balancers or cluster-internal Services are not entry points (though relayed content still reaches them; Section 7). Two verifiability caveats:

- **NodePort cannot be decided from inside the cluster.** A NodePort opens the port on every node, but whether the internet can reach it depends on node public IPs and cloud firewall rules—information the Kubernetes API does not have. The gate MUST combine cluster state with the node-level Gate 1/Gate 2 results, or fall back to an explicit, recorded operator assertion. With neither, the NodePort is treated as exposed.
- **Init containers.** In an exposed pod, `initContainers` run once, finish before the main containers start, and listen on nothing afterward, so they are not directly exposed. The exception is `restartPolicy: Always`, which turns an init container into a long-running sidecar; those are treated as primary containers.

5.5 Gate 5: Process-to-Port Refinement — Evaluating live(P)

The live(P) condition asks whether a path ends at a port something actually listens on—and its per-process refinement is where the asset tag sharpens into per-software precision. Not every package on an exposed host is itself exposed. Software MAY be excluded from *direct* exposure when it maps, deterministically, to running processes that hold no internet-exposed listening socket—`sshd` vulnerabilities on a public web server whose port 22 is locked to attributed sources under Gate 2, for example. The mapping (package manifest joined with socket-ownership telemetry) MUST be automated and reproducible per asset.

This gate has a hard limit, and it echoes the lesson of Section 2. A queue worker on a private VM with *no listening socket at all*—a thumbnail generator, a log ingester, exactly the Log4Shell shape—is still IRV for its parser flaws if it parses internet-uploaded files. "Binds no exposed port" closes only the connection channel; the content channel closes only when no internal edge delivers the flaw's trigger class—either because topology rules the path out or because verified sanitization neutralizes it (Sections 7 and 8). Gate 5 refines direct exposure; it is never a standalone NIRV call.

6 The Descriptor Space

Attack vector, protocol, and ALPN are all just *fields* of one tuple:

$$d = \langle\, \kappa,\ \tau,\ \pi,\ \nu,\ \delta \,\rangle$$

field	meaning	source
$\kappa \in \{N, A, L, P\}$	attack-vector class	CVSS **AV**
τ	transport + port (e.g. `tcp/443`)	sockets / SG / LB listeners
π	L7 protocol (`ssh`,`http`,`tls`,`dns`)	service identity / CPE
ν	protocol version / **ALPN** (`http/1.1`,`h2`,`h3`)	negotiated per hop
δ	direction (inbound-listen / outbound)	path role

The κ field is exactly the CVSS **Attack Vector** (`AV`) base metric, and its four values are CVSS's own: N = Network (remotely exploitable—the internet counts), A = Adjacent (same local network only), L = Local (needs a shell or local session), P = Physical (needs to touch the device). The other fields (τ, π, ν) are *not* in the CVSS vector—they come from the deployment and the CVE details.

Recall the two sets from Section 4: $X(v)$ is the **required surface**—the descriptors the exploit needs in order to fire—and $E(a)$ is the **exposed surface**—the descriptors the internet can actually deliver to the component. Reachability is just whether they overlap, $E(a) \cap X(v) \neq \varnothing$. Two descriptors *match* when they agree on every field; a field set to the wildcard $\star$ matches *any* value.

What the wildcard is for. Every descriptor has the five fields above, but you cannot always fill them all in—e.g. a CVE write-up may never state which protocol *version* is affected. That unknown field is set to $\star$ rather than guessed. Since $\star$ matches anything, it makes an overlap *more* likely, so a missing fact safely errs toward IRV (the fail-safe of Section 12); a field is pinned to a specific value only when there is evidence for it.

With the fields recalled—κ = attack-vector class (the CVSS `AV`), π = L7 protocol, ν = protocol version / ALPN—the one intersection test quietly does three jobs at once:

- **AV gating (the κ field) is free.** An `AV:L` (local-only) vuln needs $X(v) = \{\kappa{=}L\}$. But $E(a)$ is the *internet*-exposed surface, so it only ever contains $\kappa{=}N$ (network) descriptors. The two can never share a κ, so $E(a) \cap X(v) = \varnothing \Rightarrow$ **NIRV, regardless of how exposed the asset is.** No special case—the algebra does it. (Same for `AV:A` adjacent and `AV:P` physical.)
- **Protocol gating is the π field.** An `ssh` exploit ($X(v)$ has $\pi{=}$`ssh`) is NIRV unless `ssh` is on the exposed surface $E(a)$—even if the box exposes HTTP.
- **ALPN gating is the ν field.** An HTTP/2-only exploit ($\pi{=}$`http`, $\nu{=}$`h2`) is NIRV if $E(a)$ carries only `http/1.1`.

The π field is where the classic interior false positive dies. EternalBlue—the SMB flaw behind WannaCry—gets flagged by a scanner on an interior Windows server that internet-derived data genuinely reaches. But the flow telemetry shows that the only thing arriving on its inbound edges is PostgreSQL database traffic from the application tier: $\pi{=}$`smb` $\notin E(a)$. EternalBlue fires only when the machine *parses SMB messages*, so $X(v) = \{\pi{=}\texttt{smb}\}$, the intersection is empty, and the finding is excludable—with the flow records as the audit evidence. The exclusion polices itself in both directions. It rests on continuously observed flows: if someone later mounts a file share on that server and an SMB edge appears, the exclusion collapses on the next derivation. And it is narrow by construction: the same server's flaws in *PostgreSQL* parsing stay IRV, and an upload service that accepts arbitrary files excludes nothing, because "arbitrary files" matches every parser's trigger surface. The CVE-to-surface mapping comes from enrichment data (affected component, CPE, vendor advisory); where the required surface cannot be determined, $X(v)$ wildcards to $\star$ and the vulnerability stays IRV.

6.1 Execution Evidence: The Zeroth Factor

A descriptor intersection can only fire in code that runs. Execution evidence therefore zeroes a finding regardless of any surface computation, in two tiers:

- **The package never runs:** it is installed (so scanners flag it), but no process from it has run during the observation window—build leftovers, disabled agents, unused OS components. VEX justification: `code_not_present`.
- **The vulnerable function never runs:** the library loads, but runtime telemetry (eBPF-based library and symbol tracing, for example) shows the vulnerable function is never called. VEX justification: `code_not_reachable`.

(The corresponding CISA status-justification labels, under the companion profile's optional crosswalk, are `vulnerable_code_not_present` and `vulnerable_code_not_in_execute_path` respectively.) This evidence works for every class of flaw—injection, parser bugs, protocol bugs alike—which makes it the highest-yield exclusion to automate. It also generalizes Gate 5: Gate 5 only removes *direct* exposure for processes that listen on nothing, while execution evidence rules out triggering entirely, no matter how the data arrives.

6.2 Trigger Mechanism: Content-Triggered vs. Peer-Bound

The descriptor a vulnerability requires depends on *how its trigger arrives*, and vulnerability classes (keyed on CWE, available from NVD enrichment) split into two families:

- **Content-triggered** flaws are set off by *what the data says*, so their required descriptor rides the data—and data survives every hop. The canonical example is SQL injection (CWE-89): the attacker types a malicious string into a public search box, the web application passes that string to the internal database, and the database executes it. The attacker never connects to the database—their *text* does the work, and text survives every hop. Log4Shell worked the same way (CWE-917 expression-language injection): a booby-trapped string, logged by some service deep in the stack, triggered code execution on machines the attacker could not even name, let alone connect to. The same logic covers OS command injection (CWE-78), deserialization of untrusted data (CWE-502), memory corruption in file and format parsers (CWE-119 buffer overflow, CWE-787 out-of-bounds write—the malicious PDF or image that exploits whatever eventually opens it), XML external entity injection (CWE-611), server-side request forgery (CWE-918), and path traversal (CWE-22). **Content-triggered descriptors propagate to every downstream surface that receives the data un-neutralized**—which is exactly why the Sanitize operation of Section 7 is the one transfer function that stops them, and exactly FedRAMP's SQL-injection and Log4Shell concern.

- **Peer-bound** flaws are set off by *holding the connection*—and a connection, unlike data, cannot be forwarded. Heartbleed is the canonical example: to exploit it, the attacker's own machine had to open a TLS session with the vulnerable server and send malformed heartbeat messages over that session. regreSSHion (CVE-2024-6387) is another: the attacker must connect directly to the SSH port and race its login timer. The class covers flaws in the mechanics of the connection itself: the TLS or protocol handshake that opens it, the service's login check (CWE-287 improper authentication and CWE-306 missing authentication for a critical function, on the listening service itself), and connection-flood denial of service. In descriptor terms, the required surface of a peer-bound flaw includes "the attacker is the connection peer"—a descriptor no internal leg ever delivers, for a physical reason: when the web tier needs something from an interior server, it does not hand the attacker its connection. It opens a *new* connection of its

own—its own handshake, its own credentials—and the attacker's data rides inside only as cargo. **Peer-bound vulnerabilities are therefore excludable on assets with no reachable direct path ($R(a) = 0$)**: the only machines that can connect to an interior listener are interior systems, and none of them is an internet source. The exclusion never applies to machines the internet *can* connect to—Heartbleed on the public load balancer stays IRV.

Two cautions keep this split honest. First, stored XSS is content-triggered: the script arrives from the internet and is faithfully forwarded through the stack, even though it finally fires in a browser. "XSS is client-side, therefore excluded" is wrong. Second, not all denial of service is peer-bound: ReDoS and decompression bombs are triggered by the content of the data and follow it wherever it flows. A blanket exclusion of "XSS and DoS" is exactly the reckless evaluation `VER-EVA-ELX` warns against. Any vulnerability whose CWE is missing or fits neither family defaults to content-triggered and stays IRV.

IN PLAIN TERMS

Flaws split into two kinds by how the attack arrives. Some are set off by the *content* of the data — a poisoned string, a booby-trapped file. Those travel: whatever your systems pass along, the poison rides inside, so the flaw is in play wherever the data goes (unless something along the way provably cleans it — next section). Others require the attacker to be the one *holding the connection* — like Heartbleed. Those don't travel: your app tier calling your database opens its own fresh connection, and the attacker isn't on it. So connection-type flaws on interior machines can come off the list; content-type flaws can't be excluded by their class alone — and when in doubt, a flaw is treated as the traveling kind.

7 Where $E(a)$ Comes From: Surface Propagation

The exposed surface is the internet's full descriptor universe $D_\top$ pushed through every hop on the path to a. Each hop h_i is a **transfer function** $T_i : 2^D \to 2^D$:

$$E(a) \;=\; \big(T_k \circ T_{k-1} \circ \cdots \circ T_1\big)(D_\top)$$

IN PLAIN TERMS

Start by assuming the internet can send this component *anything* ($D_\top$ — every port, protocol, and version). Then follow the traffic through each device on the path — load balancer, firewall, proxy — and at each one, cross off whatever it blocks and rewrite whatever it changes (e.g. an HTTP/2 request the load balancer forwards as HTTP/1.1). Whatever is *still standing* when it reaches the component is its real exposed surface $E(a)$. It's like water pressure at the tap: what comes out depends on every valve between the reservoir and your faucet, not just whether the reservoir is full.

Four hop operations cover everything:

1. **Filter** — drop descriptors whose τ/π the hop does not forward. *(Gates 1, 2, 4: routing, SG CIDR scope, ingress binding.)*

2. **Translate** — rewrite a field: TLS termination π : `https`$\mapsto$`http`; NAT on τ; **ALPN downgrade** ν : `h2`$\mapsto$`http/1.1`.

3. **Authenticate** — an edge proxy (IAP, AWS Verified Access) that logs the visitor in *before* traffic reaches the component rewrites the attack-vector field $\kappa : N \mapsto A$—i.e. CVSS `MAV:A` (adjacent, not network). The κ intersection then excludes the finding automatically, with no

separate auth rule needed—provided the boundary qualifies under Gate 3's role test. Only auth enforced *ahead of* the component counts; the proxy's own pre-auth surface keeps $\kappa{=}N$ and stays IRV.

4. **Sanitize** — a component that validates or sanitizes internet-sourced input before handing data onward strips the *content-triggered* descriptors it neutralizes from the surface it propagates downstream: parameterized queries remove SQL-injection delivery (CWE-89), schema validation removes deserialization payloads (CWE-502), output encoding removes stored-XSS delivery. Unlike the first three operations, which act on network hops, this one acts at a component boundary on the *internal* legs of the path—and it is earned only by evidence (Section 8).

So the five gates are not a separate method—they are *how you compute* the first hops of $E(a)$—and the internal legs, the tier-to-tier plumbing that relays content onward, are where Sanitize applies.

Enumerating the internal legs. Which internal edges exist—which components hand data to which—is itself an evidence question, and the discipline is the same one the gates use:

- **Kubernetes estates:** the union of the flows the NetworkPolicies allow (the declared graph) and the flows actually seen by CNI or mesh telemetry, such as Cilium Hubble or Istio flow logs (the observed graph). The two kinds of evidence do different jobs. Observed flows only ever *add* edges the declared picture missed: seeing traffic proves a path exists, but not seeing traffic never proves one does not, because nothing stops a new flow from starting tomorrow. Only enforcement can rule paths out—policies that cover every workload, with default-deny behind them. Where NetworkPolicies are missing or default-allow, every edge MUST be assumed present, and the permissive posture is itself a finding.

- **Non-Kubernetes estates:** the union of the flows that firewall and security-group rules allow (the declared graph) and the flows actually seen in VPC flow logs joined with host-level socket telemetry—which process held the socket, which peers it exchanged data with (the observed graph)—giving per-process flow edges rather than bare IP adjacency. The same division of labor applies: observed flows add edges the rules missed, and only deny-by-default firewall and security-group rules can rule paths out.

Because the flow evidence is collected continuously, the edge inventory—and with it every downstream surface—rebuilds itself as the architecture changes, consistent with the automation posture of `VDR-CSO-ADT`. Three properties keep the propagation honest:

- **No edge at all means the surface is empty, wholesale.** If no permitted path—direct or relayed—leads to asset a from any entry point (an isolated batch cluster behind default-deny policy, an air-gapped management enclave, a database whose only permitted client never receives internet data), then $E(a) = \varnothing$ and *every* vulnerability on a is NIRV. The claim MUST rest on one of two things: enforcement that denies the paths (default-deny rules covering the asset, with the policy snapshots as evidence), or an explicit operator attestation that the flow map is complete and no path in it leads to the asset. Quiet telemetry alone is never sufficient: watching the flow logs for a month and seeing nothing arrive proves only that nothing arrived that month. A monthly batch job, a failover path, or a disaster-recovery runbook can produce its first-ever flow the day after the window closes.
- **An edge is not a conviction.** A live internal edge delivers descriptors, not verdicts: whether a particular flaw can fire still depends on the intersection with $X(v)$—the format arriving on the edge, the mechanism class, and any verified sanitization along the way.

- **Direction and age matter.** Edges point one way: an asset that only *pulls* data from an internet-touched source still receives that data, so its surface grows; an asset that only *pushes* data to one does not. And observed flows MUST carry a bounded observation window, so edges that no longer exist age out of the inventory.

IN PLAIN TERMS

To know what the internet can deliver to an interior machine, follow the data, not the network diagram: every hand-off between your own systems — requests, queued messages, uploaded files, replication — carries the internet's content one hop further, minus whatever each hop provably blocks, rewrites, or cleans. A machine drops out entirely, flaws and all, only when something actually rules every path out: deny-by-default rules that cover it, or the operator's signed statement that the flow map is complete and no path leads there. "We watched for a month and saw nothing" is not enough on its own — quiet logs prove the past, not the future.

8 Sanitization as Surface Termination

Production applications do not pass raw internet input to their datastores and workers. They bind query parameters instead of concatenating strings. They validate uploads against schemas, check types and bounds, encode output, reject what does not parse. This is not a security exotic—it is what competent software does, taught in every secure-coding curriculum and enforced by every mature code-review culture. The maximalist reading of `FRD-IRV` treats all of that engineering as if it did not exist: data flowed downstream, therefore every downstream flaw is reachable. This section makes the engineering count—as *evidence*, not as assumption.

The mechanics come straight from Section 7. A component that sanitizes internet-sourced input before handing data onward applies a Sanitize transfer function on the internal leg: the content-triggered descriptors it neutralizes are removed from every downstream component's exposed surface E. The granularity is per vulnerability class, and that precision matters in both directions. Parameterized queries strip CWE-89 delivery from the edge to the database—and say nothing about CWE-502 deserialization on the same edge. Each class's exclusion is earned separately, which is what keeps the claim honest: the evidence establishes the specific neutralization, not a general aura of carefulness.

> **Exclusion criterion.** A content-triggered vulnerability v (class c) on downstream asset a may be excluded from transitive reachability if and only if every internal edge that carries internet-derived data toward a passes through a component whose handling of that data neutralizes class c, as verified by evidence: code-review records covering the input-handling paths, SAST results showing the class is guarded on those paths, DAST results or unit/integration tests exercising the sanitization against class-c payloads. Collecting and validating that evidence is assessment work: the 3PAO gathers it during assessment from the provider's development and continuous-monitoring artifacts, and the authorizing agency or FedRAMP may request the same evidence at any time.

This is not an invented discount. The `VER-EVA-EIR` interception note quoted in Section 2 names the preventions that end reachability—"intercept, inspect, filter, *sanitize*, reject, or otherwise deflect"—and concludes that "once this prevention is in place the vulnerability should no longer be considered an internet-reachable vulnerability." Sanitization is on FedRAMP's own list, and the note's conclusion is FedRAMP granting exactly this exclusion. What the rules leave open is the evidence bar, and that is this paper's contribution: the claim is auditable, not asserted. "We

sanitize our inputs" earns nothing; a code-review record over the data-access layer, a SAST run scoped to the input-handling paths, a test suite that fires class-c payloads at the boundary and shows them neutralized —each is an artifact an assessor can examine and re-run.

The engineering is not merely customary; in a FedRAMP system it is required. Input validation is a baseline control: `SI-10` (Information Input Validation) applies to every offering at Moderate and above — Certification Classes C and D under the 2026 rules — and obligates the provider to check the validity of its information inputs, with the implementation described in the SSP and examined at every assessment. That moves the exclusion criterion above from hopeful to expected: the sanitization evidence this section asks for is not a new artifact class invented for reachability, it is the proof a provider already owes for a control it already claims, put to sharper, per-finding use. The connection runs both ways. Where the `SI-10` implementation is real, its evidence trail — the implementation statement naming the validation points, the SAST results and test suites behind it, often already organized in the provider's GRC platform — is exactly what the exclusion consumes. And where a component processes internet-sourced input with no `SI-10` story at all, the conservative default is the least of the provider's problems: the finding stays IRV, and the missing control is an assessment finding in its own right.

Nor does the evidence require new machinery, because most of the stream already flows. FedRAMP continuous monitoring requires regular web-application scanning—DAST by another name—and those scans do precisely this work: they fire injection, traversal, and malformed-content payloads at the running application and report what got through. A clean recurring web-application scan over the endpoints that feed a downstream tier *is* sanitization evidence for the classes it exercises, refreshed on the cadence FedRAMP already mandates. SAST and unit tests ride the provider's existing development lifecycle the same way. The assessor's job is collection and judgment—scoping which scans and tests cover which internal edges —not the invention of a parallel documentation regime. And the same economy says where the scrutiny should concentrate: on the components that process data from *unauthenticated* users. The login page, the public API, the upload endpoint, and the parsers their raw input feeds see the internet's content before any sanitization has had its chance; the tiers behind them see only what survives. An assessment that verifies the sanitization at that first rank of components has verified the estate where it matters most.

The fail-safe holds with no exceptions. No evidence, no exclusion: an undemonstrated sanitization claim leaves the descriptors propagating intact, and the finding stays IRV. Evidence gaps widen the IRV set, never shrink it. The claim also carries a revocation discipline: sanitization evidence attaches to the code paths it examined, so a major refactor of the input-handling tier re-opens the question, the same way a perimeter change re-opens a WAF claim. Continuous integration makes this cheap—the unit and integration tests that demonstrate the sanitization run on every build, so the evidence re-validates itself at exactly the cadence the code changes.

One line must be drawn carefully: this section is about sanitization *in the application's own code*, not at the perimeter. A WAF also sanitizes—at a bolt-on device in front of the stack, with failure modes (signature bypass, rule toggles, detection-only mode) that Section 13 examines in detail, which is why that section recommends attesting WAF mitigations rather than reclassifying on them. In-code sanitization is different in kind: it is the component's own behavior, shipped with the application, versioned with it, and exercised by its test suite. When verified, it earns the NIRV call directly.

IN PLAIN TERMS

Your application almost certainly cleans what the internet sends before passing it along — that is what parameterized queries and input validation are. FedRAMP's own rules say a dangerous payload stopped before the vulnerable code sees it makes the flaw not internet-reachable, and "sanitize" is on FedRAMP's list of ways to stop it. Input validation is not optional either way: control SI-10 in the FedRAMP baseline already requires it, so the proof of cleaning is proof you already owe. So the auditor's job is not to presume every downstream flaw reachable — it is to collect the proof of cleaning: the web-application scans FedRAMP already requires, the code reviews, the tests. Where the proof holds, the downstream flaw comes off the urgent clock, and the audit's attention goes where it should — the components that handle input from strangers who never logged in. Where the proof isn't there, the flaw stays on the clock: the cautious default always wins a tie.

9 The Steady State: Reachability Converges to Accessibility

Put the pieces together and the operational picture simplifies. The fail-safe says a finding stays IRV absent sanitization evidence—but the evidence collection is not an optional extra a provider might skip. The 3PAO collects it during assessment, and the recurring web-application (DAST) scanning that continuous monitoring requires regenerates it on a standing cadence, exercising the live application's input handling against exactly the payload classes the exclusions depend on. A conforming provider therefore operates with *standing* verification that its input-handling tiers neutralize the common content-triggered classes. In that steady state, the IRV set converges to the internet-accessible set: the entry points themselves, and the components that process unauthenticated internet input before sanitization has had its chance. For most reporting scenarios, the two lists coincide—not because reachability and accessibility mean the same thing (they do not, and Section 2 keeps the vocabulary apart), but because the standing evidence keeps closing the gap that the transitive definition opens.

The convergence is conditional, and the condition is exactly what threat intelligence monitors. Every sanitization exclusion is a claim about *known* trigger techniques: parameterized queries defeat the known forms of SQL injection, schema validation defeats the known deserialization gadgets, the DAST scan fires the payloads the industry knows to fire. A zero-day that defeats the known techniques—one that passes most sanitization approaches, WAF configurations, and other protections at once—dissolves those claims wholesale. Log4Shell is the canonical case. Before its disclosure in December 2021, no mainstream sanitization guidance treated `${jndi:...}` inside a loggable string as dangerous, so no sanitization evidence collected before that day said anything about it; the moment the technique published, every downstream component whose logs could carry attacker-controlled strings became reachable in exactly the way FedRAMP's definition contemplates. That is what the broad wording of `FRD-IRV` is for: it describes the posture a provider must be able to *snap to*, not the posture it lives in.

The obligation that follows is substantive, not clerical. Sanitization-based NIRV calls are revocable, and the revocation trigger MUST be wired to the provider's threat-intelligence inputs—CISA KEV additions, vendor advisories, EPSS spikes, exploit publications. Whether the provider then re-labels the affected findings IRV in its internal tracking is its own documentation choice; the model does not quibble over ledger mechanics. What the event demands is due diligence: sweep for indicators of compromise (IoC) across every component the suspect data touches, implement the known mitigation techniques, and monitor continuously to see whether *additional* mitigations are required until a patch exists and is applied—because under a live technique, the mitigations themselves evolve. Log4Shell made that concrete: the first WAF rules blocking `${jndi:ldap://...}`

were bypassed within days by obfuscated variants, and every vendor spent December 2021 rewriting patterns in a game of whack-a-mole (Section 13 tells the story)—when no patch is in place, yesterday's mitigation is only a hypothesis about tomorrow's traffic. Each suspended exclusion is re-earned only with new evidence that the sanitization holds against the new technique—an updated guard, a test that fires the new payload and shows it inert, a fresh scan—and the re-evaluations and mitigations are reported under `VER-RPT-VDT` like any other. This is the same revocation discipline the model already applies when a refactor touches the input-handling tier or a perimeter change touches a WAF claim; the only difference is that the trigger arrives from outside the provider's walls.

IN PLAIN TERMS

Day to day, the internet-reachable list ends up looking like the internet-facing list — because the auditors and the required scans keep standing proof that the cleaning between your front door and your back tiers actually works. But that calm depends on attackers using known tricks. When a new trick drops that beats everyone's cleaning — Log4Shell was exactly this — assume it travels: treat the affected flaws as urgent everywhere the data flows, hunt for signs of compromise, put mitigations in, and keep adjusting them — Log4Shell's first WAF rules were dodged within days, and defenders had to keep rewriting them until patches landed. How you label all this in your own tracker is your call; doing the work isn't. FedRAMP's sweeping definition is written for that day, not for every day.

10 The Model, Worked

1. **`AV:L` on an internet-facing web host.**
   $X(v) = \{\kappa{=}L\}, \quad E(a) \subseteq \{\kappa{=}N\} \quad \Rightarrow \quad \varnothing \Rightarrow$ **NIRV**.

2. **`sshd` CVE; public host; SSH open only to the corporate network.**
   The host has a public IP and serves HTTPS to the world on `tcp/443`, but its security group allows `tcp/22` *only* from the corporate range `w.x.y.z/24`. Building $E(a)$: start from $D_\top$ (all traffic); the firewall hop drops `tcp/22` for every source outside `w.x.y.z/24`—which is the entire internet—while `tcp/443` survives. So the SSH descriptor is not on the exposed surface: $\pi{=}\texttt{ssh} \notin E(a)$.
   The exploit needs SSH: $X(v) = \{\pi{=}\texttt{ssh}\}$. Since $E(a)$ has no `ssh` entry, $E(a) \cap X(v) = \varnothing \Rightarrow$ **NIRV**—even though the box itself is plainly "internet-facing" (its web port is wide open). The reachable *surface* and the reachable *box* are different things, which is the whole point of scoring per finding.

3. **HTTP/2 "rapid-reset"-style CVE; backend behind a load balancer that speaks `h2` to clients but `http/1.1` to the backend.**
   The load balancer terminates the client's HTTP/2 connection and opens a fresh `http/1.1` connection to the backend behind it. Building $E(\text{backend})$: the load-balancer hop rewrites the version field, $\nu : \texttt{h2} \mapsto \texttt{http/1.1}$, so `h2` never reaches the backend—$\nu{=}\texttt{h2} \notin E(\text{backend})$.
   The exploit needs HTTP/2: $X(v) = \{\nu{=}\texttt{h2}\}$. No overlap $\Rightarrow$ **NIRV for the backend**, because the flaw can only be triggered with a protocol version the backend never actually receives.
   What about the load balancer itself? It *does* terminate `h2`, so for that component $\texttt{h2} \in E(\text{lb})$ and the same CVE is **IRV**. But that is only *your* finding to remediate if you run the load balancer's software (e.g. a self-managed nginx/Envoy). On a **cloud-managed** load balancer (AWS ALB, GCP external LB, Azure Front Door) the data-plane software is the provider's responsibility, not yours—so it never appears in your scan in the first place. Either way the per-component split is correct: the flaw lives where `h2` is actually spoken.

Precondition: the backend NIRV conclusion holds only if the load balancer is the **sole path** to that backend port—exactly what a backend security group locked to the load balancer's source guarantees. If the backend also accepts a *direct* path on that port, an attacker can send `h2` straight to it, so $\texttt{h2} \in E(\text{backend})$ (the surface is the *union* over all paths) and the finding is IRV again.

4. **SQL injection on a private-subnet database behind the application tier.**
   FedRAMP's canonical case. The database has no public IP, no internet-gateway route, and no fronting load balancer: $R(\text{db}) = 0$, so no connection channel exists. But the application tier receives internet requests and queries the database with data derived from them, so under the maximalist reading every CWE-89 finding on the database is IRV.
   Now apply the Sanitize hop. The assessment verifies that every code path from request data to SQL goes through parameterized queries: code review of the data-access layer on record, SAST clean for CWE-89 on those paths, and the recurring continuous-monitoring web-application scans firing injection payloads at the API and showing them handled inert. The internal edge $\text{app} \to \text{db}$ therefore strips the CWE-89 descriptors: $X(v) \cap E(\text{db}) = \varnothing \Rightarrow$ **NIRV**, with the collected evidence bundle as the audit artifact.

   The precision cuts both ways. The same database's CWE-502 finding in a replication listener is untouched by the CWE-89 evidence and stays IRV unless separately excluded. And where the evidence cannot be collected, the maximalist answer stands: IRV, full stop—not as a penalty, but as the fail-safe doing its job.

11 Plugging into PAIN

The finding-level predicate drops straight into the timeframe machinery of the companion memo (*A Deterministic, CVSS-Environmental Method for FedRAMP Rev5 VDR/VER Vulnerability Prioritization*). Replace any scalar asset tag ("IRV = asset is internet-reachable") with $\text{IRV}(v, a)$; nothing else in the PAIN math changes:

$$\text{column} = \begin{cases} \text{LEV+IRV} & \text{LEV} \ \wedge \ \text{IRV}(v,a) \\ \text{LEV+NIRV} & \text{LEV} \ \wedge \ \neg\,\text{IRV}(v,a) \\ \text{NLEV} & \neg\,\text{LEV} \end{cases}$$

IN PLAIN TERMS

This just picks which column of the remediation-deadline table you land in. If the flaw isn't likely to be exploited (NLEV), you get the relaxed deadline. If it is likely exploited, the question becomes *is this specific finding internet-reachable?* — yes gives you the fastest clock (LEV+IRV), no gives you the middle one (LEV+NIRV). The only change from an asset-tag world is that "internet-reachable" now means the *finding*, not just the box it sits on.

12 Determinism and the Fail-Safe

The model is evaluated coarse → fine; each field is a further exclusion opportunity with a different evidence maturity:

field	deterministic today?
κ (AV)	**Yes** — in the CVSS base vector, machine-readable
τ, π (port/proto exposed)	**Mostly** — SG rules, LB listeners, sockets, CPE
π, ν that the *vuln* needs	**Often not** — buried in prose; needs enrichment/LLM
sanitization (internal edges)	**Earned** — code review, SAST, DAST, tests; assessor-collected

Fail-safe rule. Unknown field $= \star \Rightarrow$ contributes a match $\Rightarrow$ defaults to **IRV**.

You earn **NIRV** only with *positive evidence* that the required surface is not exposed.

IN PLAIN TERMS

When in doubt, assume the worst. If you don't know a detail (say, whether the load balancer downgrades the protocol, or whether the app really parameterizes its queries), treat it as "could be reachable" and keep the tight clock. You're only allowed to relax to the slower internal clock when you can *prove* the attacker's required path isn't actually open. That way a gap in your data never accidentally makes a vulnerability look safer than it is.

So `AV:L` → NIRV today (free). SSH-not-exposed → NIRV today (SG + sockets). The ALPN-downgrade and sanitization exclusions → NIRV *when* you can evidence the upstream protocol or the neutralizing code path, and until then they safely stay IRV. Finer layers tighten the result as enrichment and evidence mature, and never silently under-classify—consistent with the companion memo's existing rule that *missing metadata must not lower PAIN*.

13 Perimeter Mitigation: Permitted Downgrade, Recommended Attestation

One design question remains: what should happen to a vulnerability's IRV status when a Web Application Firewall rule—or a comparable payload-inspecting control, such as a virtual patch or an OWASP core-rule-set deployment—verifiably blocks the traffic that triggers it? There are only two choices: *reclassify* the finding NIRV, or *keep it IRV* and publish a signed Vulnerability Exploitability eXchange (VEX) attestation that it is mitigated at the perimeter. The conclusion of this section, up front: **FedRAMP permits the reclassification; we recommend the attestation.**

IN PLAIN TERMS

FedRAMP does allow you to stop counting a flaw as internet-reachable once a WAF rule verifiably blocks the payloads that trigger it. We recommend not taking the discount: WAF rules are bypassed, tuned, and toggled off far more easily than networks are re-plumbed. Keep the finding classified reachable and attach a signed, machine-readable note that it is mitigated at the perimeter — your dashboards keep showing the true residual risk, and the auditor gets a stronger artifact. One thing no WAF and no reclassification can move: known-exploited (KEV) flaws must still be *fixed* by CISA's due dates.

FedRAMP permits the downgrade. It is tempting to assume that reclassifying a WAF-shielded vulnerability as NIRV would itself draw audit findings. The launched rules say otherwise. The `VER-EVA-EIR` interception note quoted in Section 2 states that once payload interception "is in place the vulnerability should no longer be considered an internet-reachable vulnerability," and

a WAF rule that verifiably intercepts the triggering payloads before the vulnerable resource processes them *is* such a prevention. **FedRAMP explicitly permits the downgrade.** Note that the note's verb list—"intercept, inspect, filter, sanitize, reject, or otherwise deflect"—is deliberately generic: the WAF is the canonical member of the class, not its boundary. Any control that verifiably stops the triggering payload *before the vulnerable resource processes it* qualifies on the same terms—an API gateway enforcing a request schema, content disarm and reconstruction on uploaded files, a mail filter stripping the attachment type. Everything in this section applies to that whole perimeter class. Sanitization *in the application's own code* is strong enough to carry its own section (8) and, when verified in assessment, earns the exclusion directly rather than through this section's attestation posture. The rules also lower the stakes: FedRAMP formally separates mitigation from remediation (`VDR-CSO-RES`: "a fully mitigated vulnerability will still exist (with negligible risk) until it has been remediated"; see `FRD-PMV`, `FRD-FMV`), and the `VDR-TFR-PVR` timeframes are satisfied by mitigation to a lower PAIN rating. A WAF virtual patch is a legitimate partial or full mitigation on its own, with or without any reclassification.

How an interception satisfies the clock. There is a drafting gap here worth closing, because it will be argued in assessments. `VDR-TFR-PVR`'s satisfaction language speaks of mitigating "to a lower Potential Agency Impact N-rating"—a *row* move—while a reachability downgrade moves the *column* and touches no PAIN rating at all. Read hyper-literally, the WAF that FedRAMP itself calls a mitigation could never satisfy the timeframe. The definitions close the gap: `FRD-PMV` defines a partially mitigated vulnerability as one whose "likelihood or Potential Agency Impact N-rating has been reduced," and `FRD-FMV` extends the same pair "until either are negligible." Interception is a *likelihood* reduction. A rule that verifiably blocks every variant of the triggering payload drives the likelihood of internet exploitation to negligible, making the finding *fully mitigated* under `FRD-FMV`—and a fully mitigated vulnerability leaves the timeframe treadmill for routine operations under `VDR-TFR-RMN` (KEVs excepted, below). Anything short of that is a partial mitigation on the likelihood axis: the finding is re-evaluated, lands in the NIRV (or NLEV) column, and continues on that longer clock, with the change reported under `VER-RPT-VDT`. The same logic is what lets *any* attested control in this section count as mitigation at all: the lever is the likelihood half of FedRAMP's own mitigation definition, not a PAIN reduction.

We recommend the attestation anyway. The reasons are practical, not regulatory:

- **WAF exclusions are fragile; topology exclusions are not.** Log4Shell showed why in real time. Within days of the first WAF rules blocking `${jndi:ldap://...}`, attackers published obfuscated variants—nested lookups like `${${lower:j}ndi:...}`—that sailed straight past the patterns, and every WAF vendor spent December 2021 chasing them. That is the difference between the two exclusions in one incident: a WAF exclusion is a bet that a signature matches every variant of the exploit an adversary can invent; a topology exclusion—granted only when deny-by-default rules or an attested-complete flow map show no path from any internet entry point to the asset (Section 7)—is a bet that an attacker cannot send data over a connection that does not exist. Nobody obfuscates their way through a missing path. The WAF bet also loses quietly to your own operations—a rule dropped to "detection-only" during an incident, a CDN migration, an emergency toggle each silently voids the downgrade—so reclassifying on WAF evidence means re-validating the call on every perimeter change. Easy to promise, hard to actually do.
- **When the WAF fails, the attested record is still true; the reclassified record is not.** Suppose the WAF is bypassed, misrouted, or toggled off. In reality both postures are now

equally exposed—the difference is what the books say. The reclassified finding still reads NIRV on a slow clock, and that statement became false the moment the control failed, with nothing in the vulnerability record to say so and nothing prompting a re-evaluation. The attested finding still reads IRV—which is still true—and its VEX statement names the exact control that just failed, so the record degrades to what it should: internet-reachable, mitigation in doubt, tight clock. That is the case for attestation in one sentence: the reachability classification never depends on the minute-to-minute health of a perimeter device; only the mitigation claim does, and that claim names precisely what to watch.

- **Attestation is better audit evidence and keeps the risk visible.** Keeping the vulnerability IRV with a signed VEX record—`state: not_affected`, `justification: protected_at_perimeter`, per the companion CycloneDX profile—hands the 3PAO a signed, machine-readable assertion naming the exact protecting control, while the provider's own dashboards keep showing the true residual risk. (A VEX document is a signed statement by the security team, not a legal instrument; its value is attribution and machine-readability, not liability transfer.) It also pre-indexes the re-triage drill of Section 9: when threat intelligence surfaces a technique that defeats a class of controls, the suspect findings are exactly the VEX statements naming that class—a query, not an archaeology project—whereas downgraded findings must first be rediscovered before they can be re-evaluated.
- **Distribution automates cleanly.** VEX attestations publish to container registries as OCI artifacts next to the images they cover, or to object storage; scanning tools fetch them at scan time and apply the mitigation status without touching the reachability computation itself.

Known Exploited Vulnerabilities bend for neither approach. `VDR-TFR-KEV` says Known Exploited Vulnerabilities SHOULD be *remediated* by the due dates in the CISA KEV catalog, "even if the vulnerability has been fully mitigated," per CISA BOD 26-04. Log4Shell—this paper's running example—is a KEV. No WAF virtual patch extends a BOD 26-04 date, and neither does an NIRV reclassification. Providers MUST track the KEV clock separately from both the reachability call and the mitigation status.

No other control changes the reachability answer. The same question will be asked about the rest of the security toolbox: does a secure-configuration baseline, an EDR agent, or runtime protection make a finding NIRV the way a WAF rule can? No. Reachability changes only when internet-derived data can no longer arrive at the vulnerable resource—by removing the data path (Sections 7 and 5.3), by neutralizing the triggering content in the application's own verified handling (Section 8), or by intercepting the triggering payload before the vulnerable resource processes it (this section's class of controls). The other controls are valuable, but they move different levers, and claiming NIRV on their evidence is a category error an assessor will catch:

- **Configuration that disables the vulnerable feature** (Log4Shell's `formatMsgNoLookups` flag), a removed package, code that never executes (Section 6.1): the flaw cannot fire at all. That is a *disposition*—`not_affected` with `requires_configuration`, `code_not_present`, or `code_not_reachable`—and the finding leaves the active queue with its PAIN retained, per the companion method memo. The payload still arrives; it just has nothing to set off.
- **Enforcing runtime protection** (RASP, a seccomp or AppArmor profile that denies the system calls the exploit needs, or an EDR in *blocking* mode): the WAF's sibling, and treated almost identically. Keep the finding IRV, attest `protected_at_runtime`, and take the mitigation credit through the likelihood clause above; where the block verifiably contains what exploitation can

accomplish—killing the process before persistence, denying the second-stage download—it is also cited evidence for lowering the Modified C/I/A metrics, and with them the PAIN rating, on the companion memo's ladder. The one thing it does not earn is the reachability downgrade, because it acts on the wrong side of the line: an interception stops the payload before the vulnerable code processes it, while a runtime control fires when exploitation is already underway—and for flaws whose damage completes during processing itself (Heartbleed's leak is already in the response, a ReDoS lockup has already happened), there is no second act left to block. Two cautions. The claim must be per-vulnerability: evidence that the blocking capability covers *this* CVE's exploitation technique, not the fleet-wide fact that an agent is installed. And an EDR in detect-and-alert mode earns none of this: noticing exploitation is response evidence, not prevention, and it changes no classification, no clock, and no disposition.

- **Hardening that shrinks the blast radius** (non-root users, read-only filesystems, egress denial, scoped credentials, compiler and OS protections): the flaw still fires, but it does less. That is the *impact* lever—the companion method memo's mitigation ladder lowers the Modified C/I/A metrics on cited evidence (`protected_by_mitigating_control`, `protected_by_compiler`)—and the finding descends to a lower PAIN rating on a longer clock.

The discipline is uniform: every control-based claim is attested in signed VEX, cites its evidence, and is re-evaluated the moment the control changes.

14 Summary and Conclusion

The FedRAMP VDR/VER rules ask a per-vulnerability question, and the answer has to discriminate: a determination that tags everything internet-reachable is as useless as one that tags nothing, and it sets documentation and remediation obligations no provider can carry. The finding-level model makes the determination a computation. The factored equation evaluates each factor from evidence: the asset tag $R(a)$ from the five gates—routing, attributed firewall sources, remote-access boundaries, container routing, and the process-to-port refinement—the exposed surface $E(a)$ from transfer functions composed along every hop, and the vulnerability's own requirements $X(v)$ from its class, protocol, and version. A finding is IRV exactly when the surfaces overlap, and every factor defaults to reachable when its evidence is missing, so gaps show up as conservatism, never as silent false negatives.

The model's practical center is the factor the ecosystem already owns. The input sanitization production applications perform every day—parameterized queries, schema validation, output encoding—is a prevention FedRAMP's own interception note names, and verifying it is the auditor's job, done with evidence streams that already flow: the web-application (DAST) scanning continuous monitoring requires, plus the code reviews, static analysis, and tests of the provider's development lifecycle. Verified sanitization terminates the reachability of the neutralized vulnerability classes downstream and concentrates assessment scrutiny where the risk lives—on the components that process data from unauthenticated users. What is not verified stays IRV; what is verified comes with artifacts a 3PAO or agency can examine and re-run. And because that verification is standing, the everyday result is proportionate: for most reporting scenarios the internet-reachable set converges on the internet-accessible set, while the full breadth of FedRAMP's definition is held in reserve for the day threat intelligence surfaces a technique that defeats the known sanitization, WAF configurations, and other protections—when the affected exclusions are suspended and the real work runs, however the provider labels it internally: hunting indicators of compromise wherever the data flows, applying and evolving mitigations, and monitoring until a patch lands and the evidence is re-earned. Where a WAF virtual patch intercepts the triggering payloads at the perimeter,

FedRAMP permits the reachability downgrade—but the stricter, more durable posture remains keeping the vulnerability IRV and attesting the mitigation with signed CycloneDX VEX, while still remediating Known Exploited Vulnerabilities on CISA BOD 26-04 due dates per `VDR-TFR-KEV`. Run the model as continuous automation and the determination rebuilds itself as the architecture changes—proportionate in what it demands, and defensible line by line under the scrutiny the new rules invite.

Glossary of Abbreviations

`3PAO`	Third-Party Assessment Organization
`ALB`	Application Load Balancer (AWS)
`ALPN`	Application-Layer Protocol Negotiation (TLS extension)
`BOD`	Binding Operational Directive (CISA)
`CDN`	Content Delivery Network
`CISA`	Cybersecurity and Infrastructure Security Agency
`CNI`	Container Network Interface
`CPE`	Common Platform Enumeration
`CVE`	Common Vulnerabilities and Exposures (vulnerability identifier)
`CWE`	Common Weakness Enumeration (vulnerability class taxonomy)
`DAST`	Dynamic Application Security Testing (web-application scanning)
`DoS`	Denial of Service
`eBPF`	extended Berkeley Packet Filter (kernel-level runtime telemetry)
`EDR`	Endpoint Detection and Response
`EKS`	Amazon Elastic Kubernetes Service
`FRD`	FedRAMP Definition (rule-ID prefix, e.g. `FRD-IRV`)
`GKE`	Google Kubernetes Engine
`IAM`	Identity and Access Management
`IAP`	Identity-Aware Proxy
`IGW`	Internet Gateway
`IoC`	Indicator of Compromise
`IRV`	Internet-Reachable Vulnerability
`JWT`	JSON Web Token
`KEV`	Known Exploited Vulnerability (CISA catalog)
`LB`	Load Balancer
`LEV`	Likely Exploitable Vulnerability
`MFA`	Multi-Factor Authentication
`NAT`	Network Address Translation
`NIRV`	Not Internet-reachable Vulnerability
`NLEV`	Not Likely Exploitable Vulnerability
`NVD`	National Vulnerability Database
`OCI`	Open Container Initiative (registry artifact format)
`OIDC`	OpenID Connect
`OWASP`	Open Worldwide Application Security Project
`PAIN`	Potential Agency Impact N-rating (N1–N5)
`RASP`	Runtime Application Self-Protection

ReDoS	Regular-expression Denial of Service
SAST	Static Application Security Testing
SMB	Server Message Block (Windows file-sharing protocol)
SQL	Structured Query Language
SSH	Secure Shell
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
VDR	Vulnerability Detection and Response (FedRAMP rule set)
VER	Vulnerability Evaluation and Reporting (FedRAMP rule set)
VEX	Vulnerability Exploitability eXchange
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WAF	Web Application Firewall
XSS	Cross-Site Scripting
XXE	XML External Entity (injection)